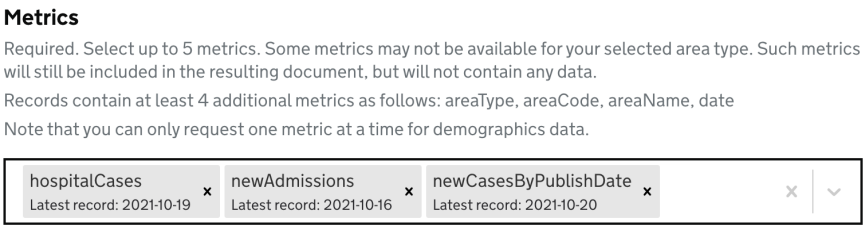
Using Python and SciKitLearn to Predict the Behaviour of People during COVID

This blog follows on from a blog we did a few months ago entitled COVD-19 Data Analysis in Black and White with Python Pandas. In that blog we looked at two data sets that can be obtained free from public sources and how the Python Pandas library can be used to perform some simple data analytics. In this blog we take those data sets further and use several machine learning systems to create predictors for people's behaviour based on the 2020 COVID data.

The Data

As with the previous blog, we shall be using two datasets that provide information on the COVID pandemic.

The first dataset is provided by the UK Government as part of its Coronavirus (COVID-19) data provision. On their web site it is possible to download data relating to the COVID pandemic using a web from. The user can select the metrics of interest, for example:



In this case the metrics represent the number of hospital cases, the number of new admissions and the number of new cases by published date.

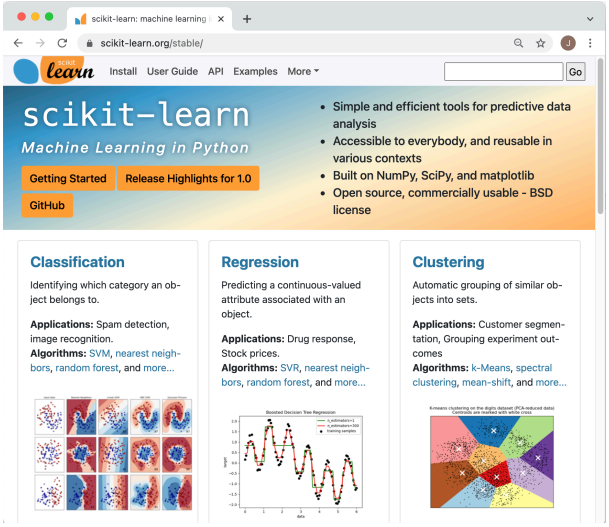
To make it easy to repeat such downloads, they provide a permanent link that allows the same data to be downloaded. The link used to download the data for this blog is:

```
https://api.coronavirus.data.gov.uk/v2/data?areaType=overview&metric=hospitalCases&metric=newAdmissions&metric=newCasesByPublishDate&format=csv
```

The second data set is provided by Google. It is accessible from the Google COVID-19 Community Mobility Reports site. This site provides two files that can be downloaded; the first provides a Global view of mobility while the second provides separate data files for all the regions covered by Google. This second data set is the one which was downloaded for this blog. This download is provided as a ZIP file. When unzipped it contains many different data files for different countries. For this blog the GB data file was selected.

SciKitLearn

To handle creating various machine learning algorithms / models we shall be using the SciKitLearn library.



This is a Python library that is built on top of NumPy, SciPy and MatPlotLib and provides implementations for a wide range of machine learning approaches including those used for classification, regression, clustering etc. SciKitLearn is very widely used, is open source and can be used for commercial applications. It is of course not the only library available and other widely used Python machine learning libraries are TensorFlow and PyTorch.

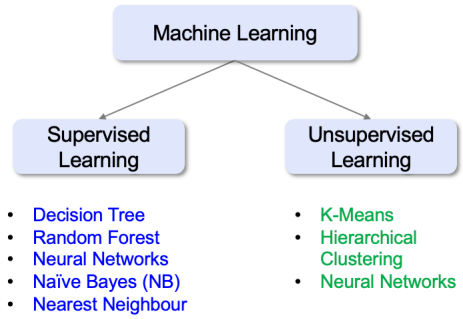
To install SciKitLearn you need to add it to your Python interpreter. There are several ways in which this can be done depending on how your environment is set up. Two common approaches are to use `pip` or `conda` (depending on how you are manging your Python environments). Pip is a tool used to install Python packages and is provided as part of Python. Conda, another package management tool, is part of Anaconda and is a popular choice for data scientists using Python. Both can be used to install Pandas as shown below:

```
pip install scikit-learn
```

```
conda install –c conda-forge scikit-learn
```

Learning Systems

There are a range of Machine Learning algorithms (often referred to as models) provided by SckKitLearn that can be used to predict future patterns based on past data. These can be divided up into supervised and unsupervised learning systems.



A supervised learning system involves teaching a system, using data that is tagged or marked with known results. By contrast an unsupervised learning system involves the system itself identifying patterns or clusters with the data that it is presented with. The term unsupervised is intended to indicate that there are no known (or at least provided) correct answers.

In this blog we will be focussing on the use of supervised learning with K-Nearest Neighbour, Decision Trees and Random Forest machine learning algorithms.

The Problem

The aim of this work is to generate a system that can predict the percentage change in *retail and recreational mobility* based on the number of hospital cases, new hospital admissions and covid numbers.

To do this we need to take the data provided by the UK governments COVID site and the Google mobility data and merge it together into a single data set. This process was described in the previous blog and is therefore not repeated here (however the program used to merge the data can be found here within a GitHub repository).

Having obtained the relevant data we now need to split that data into a set of data to be used for *training* the learning system and a set of data that we can use to *test* its performance. To do this we use the `train_test_split()` function provided by the SciKitLearn library. This function randomly splits the dataset into train and test subsets. Using the `test_size` parameter it is possible to indicate the percentage split between training and testing. For example, using `test_size=0.2` indicates that the train set will hold 80% of the data and the test set will hold 20% of the data.

This is illustrated below. Note that `df` holds a Pandas DataFrame and that `train_test_split()` returns two DataFrames held in `train` and `test`:

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv(merged_covid_data.csv')
train, test = train_test_split(df, test_size=0.2)
```

Based on this split we can now extract the columns in the `DataFrame` that will be used as the input values (the feature set) and the output values (the target variable). To do this we can indicate to the train and test DataFrames

```
FEATURES_SET = {'hospitalCases',
                'newAdmissions',
                'newCasesByPublishDate'}
TARGET_VARIABLE = 'retail_and_recreation_change'

training_features = train[FEATURES_SET].values
training_target = train[TARGET_VARIABLE].values

test_features = test[FEATURES_SET].values
test_target = test[TARGET_VARIABLE].values
```

A `DataFrame` allows one or more columns to be specified using the square brackets accessor operator (e.g. `[TARGET_VARIABLE]`). These can then be converted into a simple NumPy `ndarray` using the values attribute. These `ndarrays` can then be used with the various learning systems for training and testing purposes.

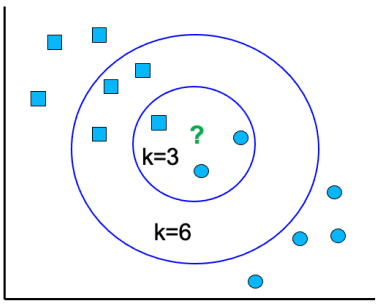
Using Regression Supervised Learning Systems

In this blog we will be using three *regression* supervised learning systems. We are using Regression Models as they can be used to predict a continuous quantity output whereas machine learning classification algorithms predict a set of discreet values (see Difference Between Classification and Regression in Machine Learning for more information on this).

We will be using the K-Nearest Neighbour Regressor, the Decision Tree Regressor and the Random Forest Regressor from the SciKitLearn library.

K-Nearest Neighbour Regressor

K-Nearest Neighbour (or KNN) is one of the simplest and most widely used algorithms within the Machine Learning world. KNN uses past data to predict the classification of new data based on a similarity measure (the nearest neighbour aspect). The closer a new data point is to a group of related data points then the more likely it is to be classified as being of that type of data. The *k* value in the nearest neighbour algorithm indicates how may nearby data points should be considered, to determine the classification of the new value. For example:



In the above diagram, if *k* (the number of nearest neighbours to be considered) is 3 then the ? data point will be classed as a circle. However, if *k* is set to 6, then the ? data point will be classed as a square.

SciKitLearn provides the `KNeighborsRegressor` class as an implementation of a KNN regressor. This class can be instantiated with the value of *k* as a parameter (called `n_neighbours`). For example:

```
knn_model = KNeighborsRegressor(n_neighbors=3)
```

The above line of code creates a new KNN model instance initialised with the value of *k* (`n-neighbours`) set to `3`.

We can now train the KNN model object using the training feature set and the training target output, for example:

```
model.fit(training_features, training_target)
```

Once this has run the KNN model object has been *taught* based on the training data.

The question now is how well has this worked?

There are two metrics commonly used to assess how *good* a learning system such as the KNN regressor is; these are RMSE (Root Mean Square Error) and R-Squared:

RMSE is a standard way to measure the error of a model in predicting data. It is used in Machine Learning as a way of evaluating the accuracy (or usefulness) of a trained model. In general, the smaller the RMSE value the better the result. Of course, *small* is a relative term, but when an RMSE value is compared with the RMSE values obtained for alternative models on comparative datasets, it can be used as a relative guide to the utility of one model against another.

R-Squared is a statistical measure of how close the data is to the fitted regression line. In other words, it indicates how well the model fits the data as a percentage, with Zero indicating a very poor fit and 100% indicating a perfect fit. Thus, the higher the R-Squared value the better.

We can thus use these metrics to assess how well our KNN model fits the data it was trained with as well as how successful it is with the test data (it has never seen before). The following code does exactly that using functions provided by NumPy and SciKitLearn:

```
print(f'KNeighborsRegressor(n_neighbors=3)')

# Determine the metrics - against the training data
pred_train_rf = model.predict(training_feature_set)
trainingRMSE =
np.sqrt(mean_squared_error(training_target_attribute,
                                          pred_train_rf))
trainingRSquared = r2_score(training_target_attribute,
                            pred_train_rf)
trainingRSquared *= 100

# Determine the metrics based on the test dataset
pred_test = model.predict(test_feature_set)
testingRMSE = np.sqrt(mean_squared_error(test_target_attribute,
                                         pred_test))
testingRSquared = r2_score(test_target_attribute, pred_test)
testingRSquared *= 100
```

The values obtained for the training and testing R-Squared values are multiplied by 100 to convert them into a normal percentage for printing purposes.

The results obtained from running the metrics against the trained KNN model are presented below:

```
KNeighborsRegressor(n_neighbors=3)
Testing KNN against Training data
Training RMSE – 7.2
Training R-squared – 91.3%
Testing KNN against test data
```

```
Testing RMSE – 11.1
Testing R-squared – 77.9%
```

In the above metrics we can see that the RMSE values are quite low, and the R-Squared values are both over 77%. However, can we do better? In the next section we will explore the Decision Tree Regressor model.

Decision Tree Regressor

A Decision Tree learning system is one that builds a single rooted decision tree, typically used to classify future values. The tree is built in general, by splitting the data at each point in the tree based on an algorithm used to determine the best way to differentiate between that data. There are several algorithms used to create a Decision Tree such as ID3, C4.5 and CART (Classification And Regression Tree).

The SciKitLearn library provides the `DecisionTreeRegressor` class. This class follows a similar pattern to the KNN Regressor. This means that we must instantiate the class and configure the regressor as appropriate. For example:

```
dt_model = DecisionTreeRegressor(max_depth=4,
                                 min_samples_leaf=0.13,
                                 random_state=3)
```

In this case we have configured the `max_depth` of the tree to be 4. We have also set the `min_samples_leaf` parameter to 0.13. This parameter is used to indicate the number of samples required before a leaf node is split into a branch (where 0.13 represents 13%). Finally, `random_state` is set to 3 to ensure that the behaviour of the tree is deterministic.

Next, we can train the DecisionTree model instance. Note that we do this in the same way as we did for the KNN model instance:

```
dt_model.fit(training_features, training_target)
```

We can now obtain the RMSE and R-Squared metrics for the `dt_model`. Again, this is done in the same way as for the KNN model. This illustrates the common pattern being provided by SciKitLearn for each of the learning systems we are working with.

The results obtained for the Decision Tree Regressor are:

```
DecisionTreeRegressor(max_depth=4, min_samples_leaf=0.13,
random_state=3)
Testing Decision Tree against Training data
Training RMSE - 11.1
Training R-squared - 81.4%
Testing Decision Tree against test data
Testing RMSE - 10.7
Testing R-squared - 80.0%
```

Interestingly these results are not generally as good as for the KNN example on the training data. The RMSE value for the KNN was 7.2 but here it is 11.1 and the R-Squared value was 91.3% where as here it is 81.4%. However, the test data results are slightly better. For the KNN the RMSE value was 11.1 but here it is 10.7 and the R-squared results for KNN were

77.9 but here they are 80%. However, overall the results are very close and given that the training and testing data is generated randomly, if we ran both experiments again we might get slightly different results. Thus overall, there is not much to separate the two Machine learning approaches.

The question is can we do better? One option would be to explore different settings for the DecisionTreeRegressor such as modifying the maximum depth of the tree or changing the percentage required before a leaf node is split into a branch. Alternatively, we could choose to try the Random Forest model (which is essentially a forest of decision trees).

Random Forest Regressor

Decision Trees are useful, but they can overfit the training data, Random Forest overcomes this by essentially creating a forest of Decision Trees. The SciKitLearn library provides the `RandomForestRegressor` class that can be used to create an implementation of a Random Forest regressor that can be trained in a similar manner to the Decision Tree and KNN models.

To create a Random Forest regressor object we use the `RandomForestRegressor` class and set the parameters used with this class as appropriate, for example:

```
rf_model = RandomForestRegressor(max_depth=4,
                                 n_estimators=500)
```

In this case we have set the maximum depth of the tree and the `n_estimators` parameter (which indicates the size of the forest).

Next, we train the random forest object as before:

```
rf_model.fit(training_features, training_target)
```

Finally, we obtain the metrics for the Random Forest model:

```
RandomForestRegressor(max_depth=4, n_estimators=500)
Testing Random Forest against Training data
Training RMSE - 7.4
Training R-squared - 92.4%
Testing Random Forest against test data
Testing RMSE - 7.4
Testing R-squared - 91.8%
```

Again, the RMSE values are not that dis-similar to those obtained for both the KNN and Decision Tree models on the training data. However, the results obtained for the test data are significantly better.

Summary of Metrics Obtained

The following table summarises the metrics obtained for the three regression models we have created.

Model	Training	Testing

	RMSE	R-squared	RMSE	R-squared
KNN	7.2	91.3%	11.1	77.9%
Decision Tree	11.1	81.4%	10.7	80%
Random Forest	7.4	92.4%	7.4	91.8%

Overall, there is not that much difference between the KNN and Decision Tree models on the testing data, but the Random Forest approach represents a significant improvement on both for the test data. We shall therefore create a Random Forest Regression based predictor for the new data obtained for 2021.

Creating the Regressor Object

We can same our predictor object (implemented using the `RandomForestRegressor` class) to a binary format file using the `pickle` package provided with Python:

```
import pickle

file = open('random_forest_regressor.pkl', "bw")
pickle.dump(rf_model, file)
```

Once we have done this we can reload and use this regressor to process new data when never required. This is done again using the Python pickle page:

```
file = open('random_forest_regressor.pkl', "br")
regressor = pickle.load(file)
```

For example, we can use it to analyse COVID data for 2021 to predict the percentage change in retail and recreational mobility.

Using the python Pandas library, we can load and configure the CSV file containing the data for 2021. This can involve ordering the data by date, removing columns not used by the regressor object as shown below:

```
df = pd.read_csv('covid_data_2021_only.csv')
df.sort_values(by=["date"], ignore_index=True, inplace=True)
# Store date column for use with output
dates = df['date']

# Drop date as now used in classifier
df.drop(['date'], axis='columns', inplace=True)

# Make sure all columns have a value even if its Zero
df['hospitalCases'] = df['hospitalCases'].fillna(0)
df['newAdmissions'] = df['newAdmissions'].fillna(0)
df['newCasesByPublishDate'] =
df['newCasesByPublishDate'].fillna(0)
```

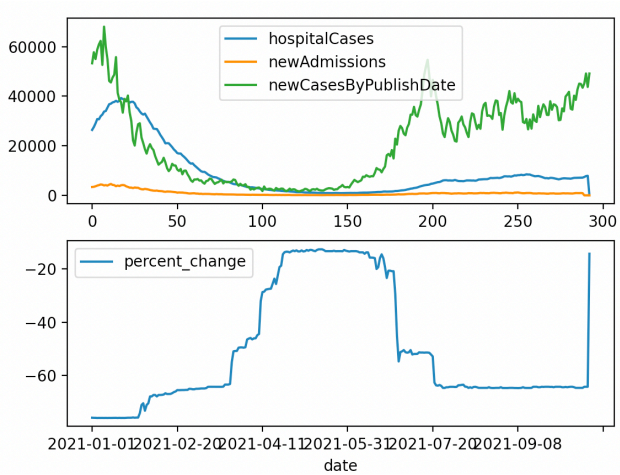
We can use the regressor object to predict the retail and recreational percentage change for the COVID 2021 data using the `predict()` method:

```
predictions = regressor.predict(df)
```

The `results` variable now holds the predictions for each row in the `DataFrame` df. However, this is not particularly useful on its own as we now need to relate the predictions back to the 2021 COVID data. We can do this by merging the with the COVID data and the dates column into a new DataFrame and graphing both the original data and the predicted percentage change. As the scales are so different, we are graphing them one above the other as shown below.

The top graph illustrates the number of new COVID cases, new hospital admissions and total hospital cases as published by the UK government across the first 10 months of 2021.

The lower graph shows the predicted percentage change in retail and recreational mobility. Note that the lower graphs X scale is negative this the lower down the graph the greater the percentage change:



These graphs show that as the number of new COVID cases decrease the Random Forest regressor predicts that the percentage change in retail and recreational mobility will decrease (notice the bottom graph has a negative scale on the left). However, as new COVID cases rise then the change in mobility negatively increases.

Note that this ignores vaccinations in 2021 as there is no data to teach the system on for 2020.

GitHub Repo

All examples used in this blog along with the sample data are available in GitHub.